365 Real Estate — Dubai Residential Price Prediction

Regression | Linear Regression vs log price Linear Regression

In [1]:
```
import pandas as pd, numpy as np, matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
import joblib, warnings
warnings.filterwarnings('ignore')
DATA_PATH='dld_transactions.csv'
raw=pd.read_csv(DATA_PATH,low_memory=False)
```

1. Original data profile

In [2]:
```
print('Shape:',raw.shape)
print('Data types:\n',raw.dtypes)
print('Missing values:\n',raw.isna().sum().sort_values(ascending=False).head(20))
print('Exact duplicate rows:',raw.duplicated().sum())
for c in ['trans_group_en','property_usage_en','property_type_en','property_sub_ty
 print('\n',c,raw[c].value_counts(dropna=False).head(20))
```

```
Shape: (172704, 34)
Data types:
 transaction_id              str
procedure_id              int64
trans_group_id            int64
trans_group_en              str
trans_group_ar              str
procedure_name_en           str
instance_date               str
property_type_en            str
property_type_ar            str
property_sub_type_en        str
property_usage_en           str
property_usage_ar           str
reg_type_en                 str
reg_type_ar                 str
area_id                   int64
area_name_en                str
area_name_ar                str
building_name_en            str
project_name_en             str
master_project_en           str
nearest_landmark_en         str
nearest_metro_en            str
nearest_mall_en             str
rooms_en                    str
rooms_ar                    str
has_parking               int64
procedure_area          float64
actual_worth            float64
meter_sale_price        float64
rent_value              float64
meter_rent_price        float64
no_of_parties_role_1      int64
no_of_parties_role_2      int64
no_of_parties_role_3      int64
dtype: object
Missing values:
 meter_rent_price       172704
rent_value             172704
building_name_en        30561
nearest_metro_en        24167
project_name_en         18992
rooms_ar                16890
rooms_en                16783
nearest_mall_en         15567
nearest_landmark_en     12086
master_project_en       10363
meter_sale_price          353
transaction_id              0
procedure_id                0
trans_group_id              0
instance_date               0
property_type_en            0
trans_group_ar              0
trans_group_en              0
area_name_ar                0
area_name_en                0
dtype: int64
Exact duplicate rows: 4704
```

```
 trans_group_en trans_group_en
Sales        113458
Mortgages     46677
Gifts         12569
Name: count, dtype: int64

 property_usage_en property_usage_en
Residential    145967
Commercial      26737
Name: count, dtype: int64

 property_type_en property_type_en
Unit        144450
Villa        19606
Land          5039
Building      3609
Name: count, dtype: int64

 property_sub_type_en property_sub_type_en
Flat                         124328
Villa                         19606
Office                        10100
Shop                           7173
Residential Land               5039
Residential / Commercial       3609
Hotel Apartment                2849
Name: count, dtype: int64

 reg_type_en reg_type_en
Existing Properties    96748
Off-Plan Properties    75956
Name: count, dtype: int64

 procedure_name_en procedure_name_en
Sell                     63423
Sell Pre-Registration    50035
Mortgage Registration    46677
Grant                    12569
Name: count, dtype: int64
```

2. Filter to residential sales and validate sizes/prices

Filters: trans_group_en is Sales; property_usage_en is Residential; positive actual_worth and procedure_area; remove implausibly tiny area below 10 m^2. We retain high prices unless inspection shows they are errors, because luxury properties can be genuine.

In [3]:
```
df=raw.copy()
df['instance_date']=pd.to_datetime(df['instance_date'],dayfirst=True,errors='coerc
for c in ['actual_worth','procedure_area','meter_sale_price','has_parking']:
 df[c]=pd.to_numeric(df[c],errors='coerce')
print('Original rows:',len(df))
df=df[df['trans_group_en'].astype(str).str.strip().str.lower().eq('sales')]; print
df=df[df['property_usage_en'].astype(str).str.strip().str.lower().eq('residential'
df=df[df['actual_worth'].gt(0) & df['procedure_area'].ge(10)]; print('After positi
print('Price quantiles:\n',df['actual_worth'].quantile([0,.01,.25,.5,.75,.95,.99,1
```

```
print('Area quantiles sqm:\n',df['procedure_area'].quantile([0,.01,.25,.5,.75,.95,
df['area_sqft']=df['procedure_area']*10.7639
df['price_per_sqft']=df['actual_worth']/df['area_sqft']
df['area_name_en']=df['area_name_en'].astype('string').str.strip().str.replace(r'\
df['sale_year']=df['instance_date'].dt.year; df['sale_month']=df['instance_date'].
print('Final filtered shape:',df.shape,'date range:',df.instance_date.min(),df.ins
display(df.head())
```

```
Original rows: 172704
After Sales filter: 113458
After Residential filter: 95845
After positive price and area >=10 sqm: 95586
Price quantiles:
 0.00    1.514824e+05
0.01    3.422588e+05
0.25    9.393269e+05
0.50    1.554230e+06
0.75    2.750140e+06
0.95    9.044311e+06
0.99    3.401123e+07
1.00    6.461994e+08
Name: actual_worth, dtype: float64
Area quantiles sqm:
 0.00       35.3100
0.01       36.7385
0.25       68.2400
0.50      104.1600
0.75      177.4075
0.95      673.3600
0.99     2814.1490
1.00    31103.5300
Name: procedure_area, dtype: float64
Final filtered shape: (95586, 38) date range: 2020-01-01 00:00:00 2025-12-31 00:00:
00
```

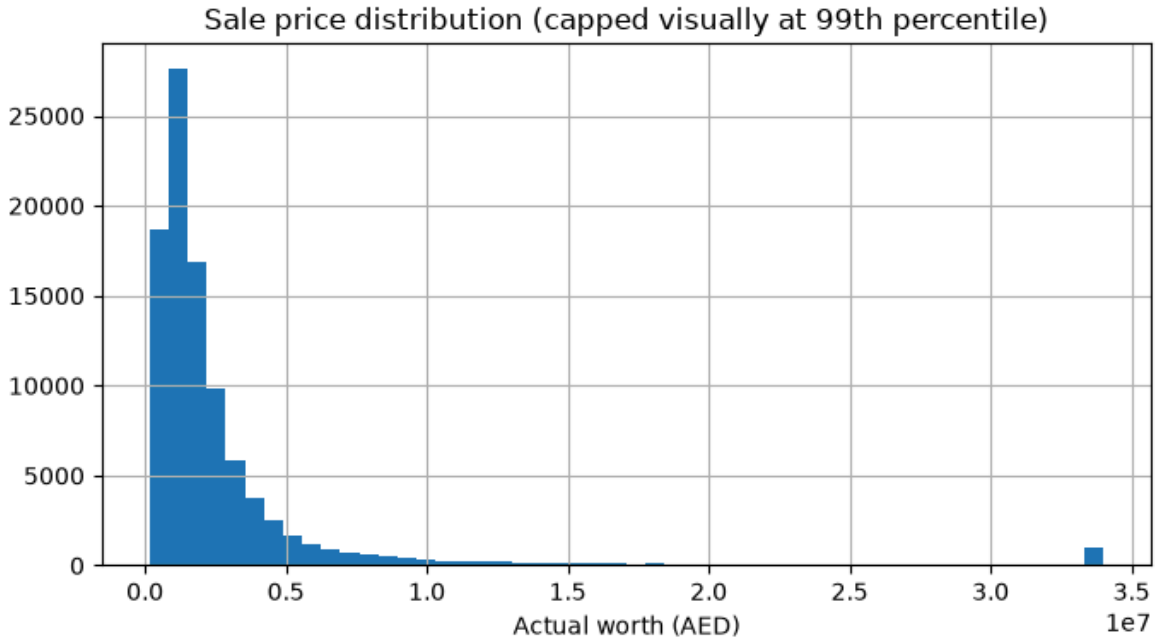
	transaction_id	procedure_id	trans_group_id	trans_group_en	trans_group_ar	procedure
2	86629276-116-2024	11	1	Sales	مبايعات	
3	43565509-159-2021	11	1	Sales	مبايعات	Sell Pre-R
5	28961253-154-2021	11	1	Sales	مبايعات	Sell Pre-R
7	55967915-249-2023	11	1	Sales	مبايعات	
8	37088223-2-2023	11	1	Sales	مبايعات	Sell Pre-R

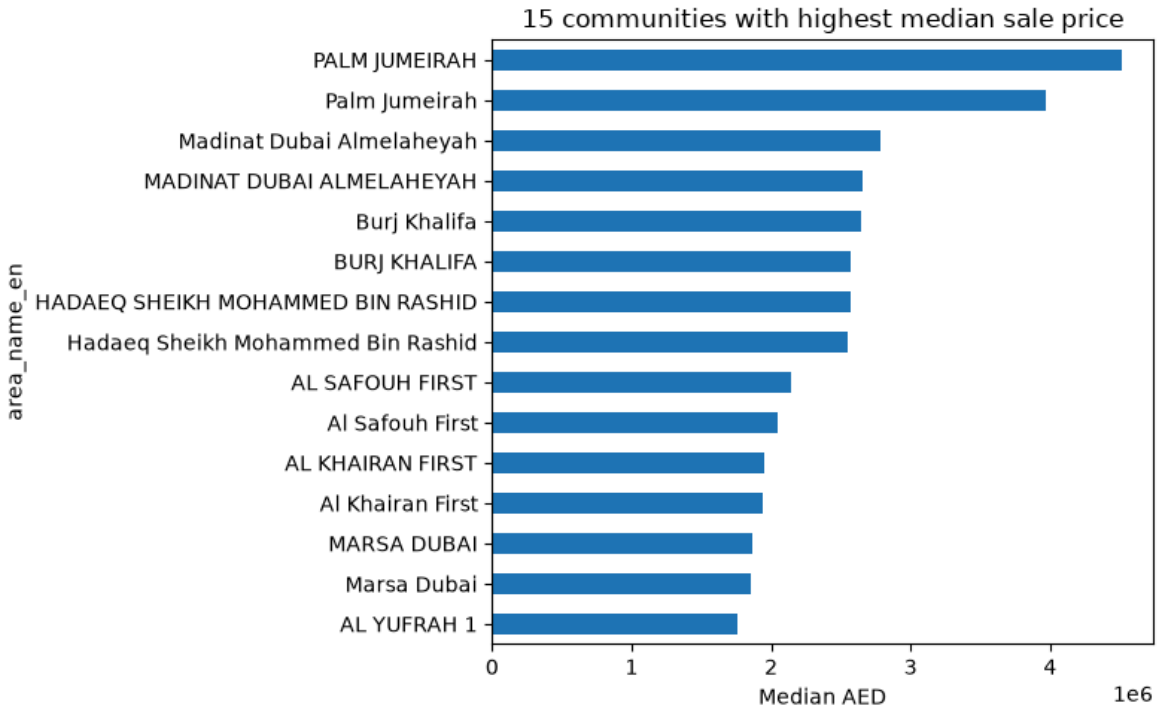
5 rows × 38 columns

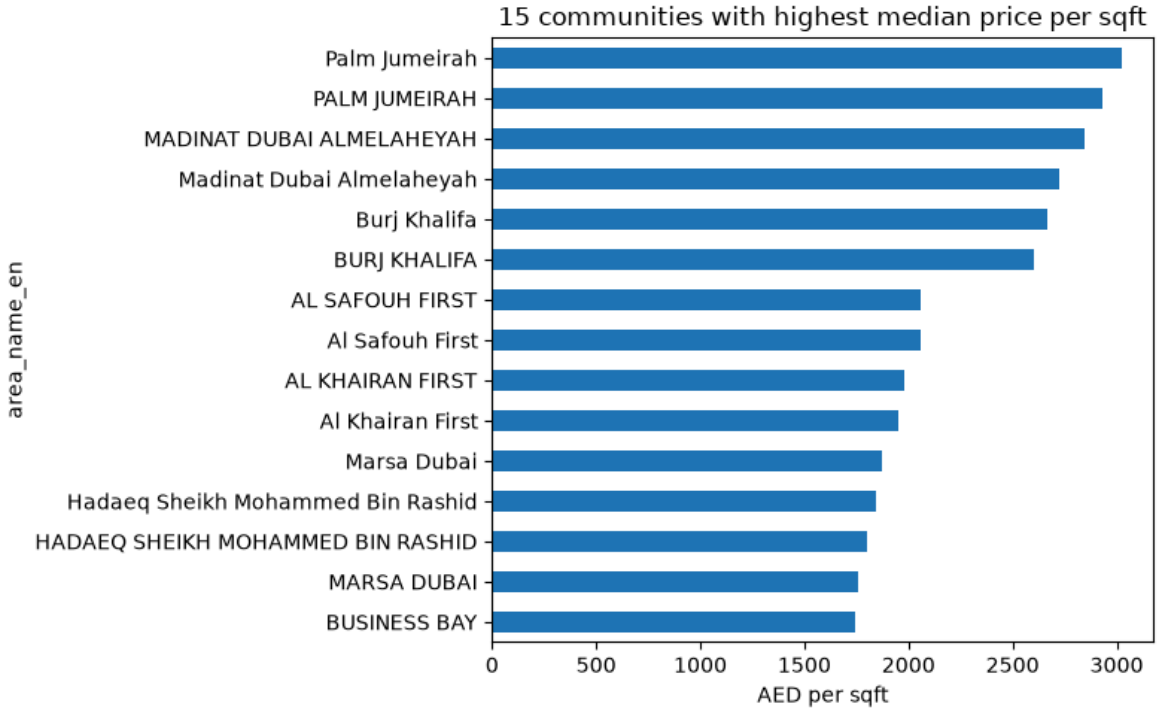
3. Exploratory market analysis (eight charts)

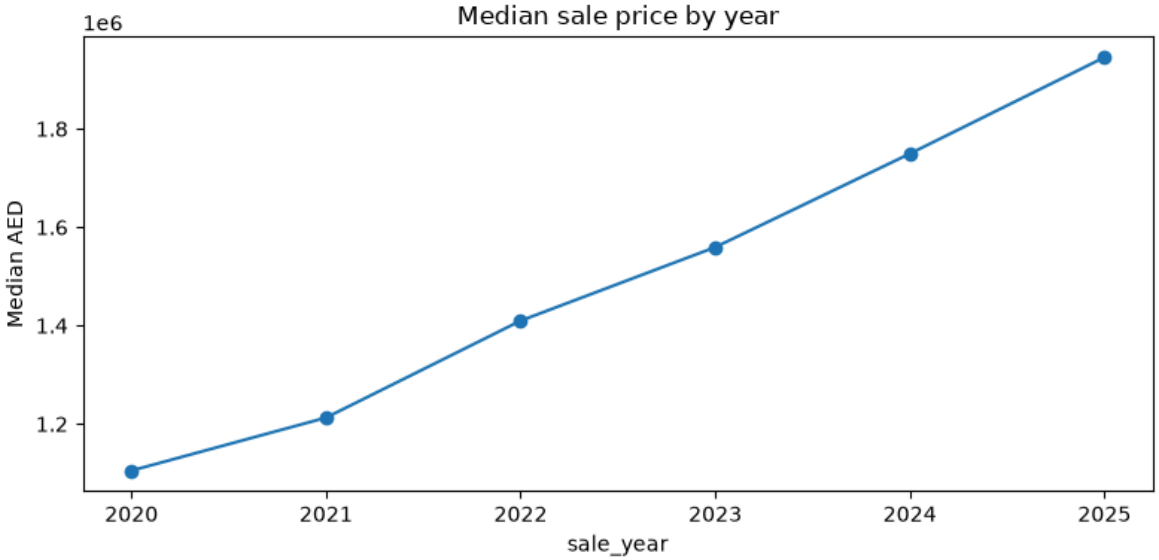
In [4]:
```
fig,ax=plt.subplots(figsize=(7,4)); df['actual_worth'].clip(upper=df['actual_worth
fig,ax=plt.subplots(figsize=(8,5)); df.groupby('area_name_en')['actual_worth'].med
```

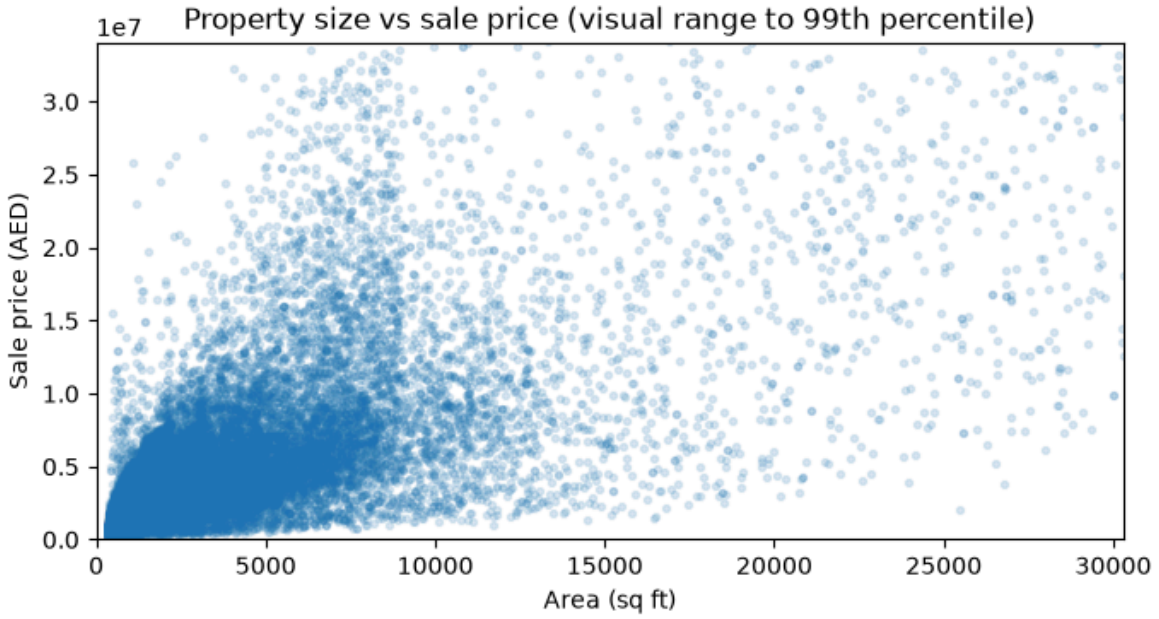
```
fig,ax=plt.subplots(figsize=(8,5)); df.groupby('area_name_en')['price_per_sqft'].m
fig,ax=plt.subplots(figsize=(8,4)); df.groupby('sale_year')['actual_worth'].median
fig,ax=plt.subplots(figsize=(7,4)); ax.scatter(df['area_sqft'],df['actual_worth'],
fig,ax=plt.subplots(figsize=(9,4)); df.groupby('rooms_en')['actual_worth'].median(
fig,ax=plt.subplots(figsize=(7,4)); df.groupby('reg_type_en')['actual_worth'].medi
fig,ax=plt.subplots(figsize=(8,4)); df.groupby('sale_month')['actual_worth'].media
```

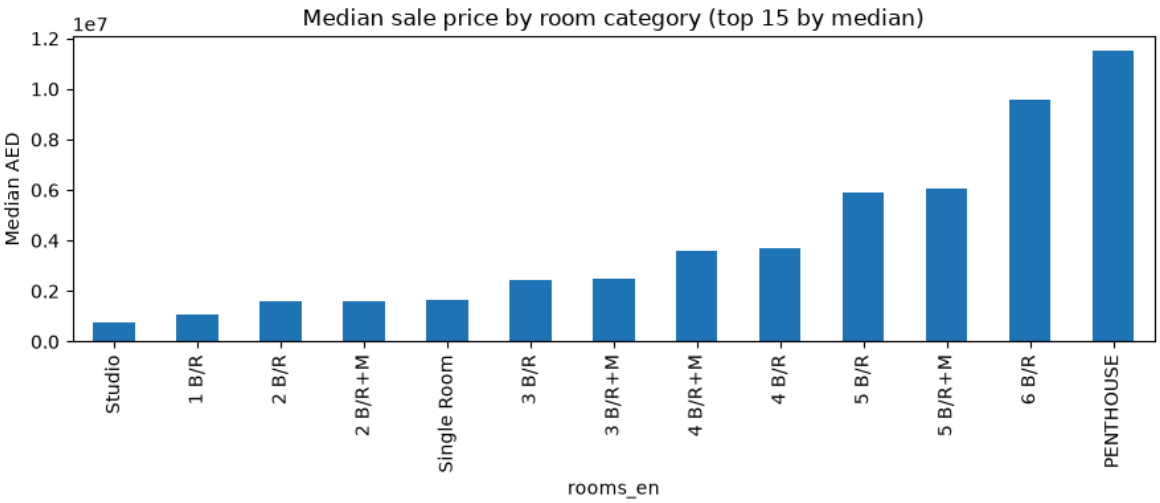


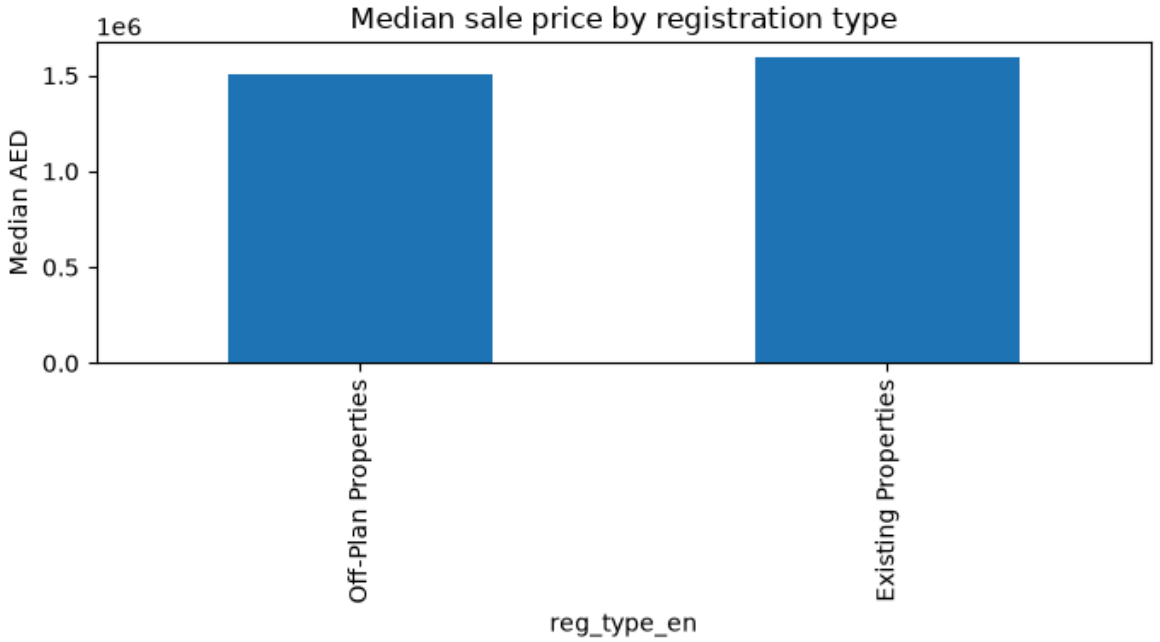



15 communities with highest median price per sqft
Palm Jumeirah
PALM JUMEIRAH
MADINAT DUBAI ALMELAHEYAH
Madinat Dubai Almelaheyah
Burj Khalifa
BURJ KHALIFA
AL SAFOUH FIRST
Al Safouh First
AL KHAIRAN FIRST
Al Khairan First
Marsa Dubai
Hadaeq Sheikh Mohammed Bin Rashid
HADAEQ SHEIKH MOHAMMED BIN RASHID
MARSA DUBAI
BUSINESS BAY
area_name_en
0
500
1000
1500
2000
2500
3000
AED per sqft


Median sale price by year
1e6
1.8
1.6
1.4
1.2
Median AED
2020
2021
2022
2023
2024
2025
sale_year


Property size vs sale price (visual range to 99th percentile)
1e7
3.0
2.5
2.0
1.5
1.0
0.5
0.0
Sale price (AED)
0
5000
10000
15000
20000
25000
30000
Area (sq ft)







4. Feature engineering and leakage review

Exclude transaction IDs, actual_worth target, meter_sale_price (derived from sale price and area), rent fields, Arabic duplicates, and post sale/administrative fields not appropriate for

pre close valuation. Convert square metres to square feet. One hot encode categorical variables.

In [5]:
```
numeric=['area_sqft','has_parking','sale_year','sale_month']
categorical=[c for c in ['area_name_en','property_type_en','property_sub_type_en',
features=numeric+categorical
model_df=df.dropna(subset=['actual_worth']).copy()
# Restrict target to positive values for log target; this is already ensured by fi
X=model_df[features].copy(); y=model_df['actual_worth'].astype(float)
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=.2,random_state=42)
pre=ColumnTransformer([('num',Pipeline([('imputer',SimpleImputer(strategy='median'
```

5. Fit raw-price and log-price Linear Regression models

In [6]:
```
raw_model=Pipeline([('prep',pre),('reg',LinearRegression())])
log_model=Pipeline([('prep',pre),('reg',LinearRegression())])
raw_model.fit(X_train,y_train)
log_model.fit(X_train,np.log(y_train))
pred_raw=np.maximum(0,raw_model.predict(X_test))
pred_log=np.maximum(0,np.exp(log_model.predict(X_test)))
def score(y,p):
 return {'R2':r2_score(y,p),'MAE_AED':mean_absolute_error(y,p),'RMSE_AED':mean_squ
results=pd.DataFrame({'Raw-price Linear Regression':score(y_test,pred_raw),'Log-pr
display(results.round(3))
chosen=log_model if results.loc['Log-price Linear Regression','MAE_AED'] < results
chosen_pred=pred_log if chosen is log_model else pred_raw
print('Selected by lower MAE on held out test set:', 'log-price' if chosen is log_
```

	R2	MAE_AED	RMSE_AED	MAPE_percent
Raw-price Linear Regression	0.761	1125998.813	3866780.787	45.789
Log-price Linear Regression	0.235	810941.676	6917240.922	20.381

```
Selected by lower MAE on held out test set: log-price
```

6. Error analysis and actual vs predicted

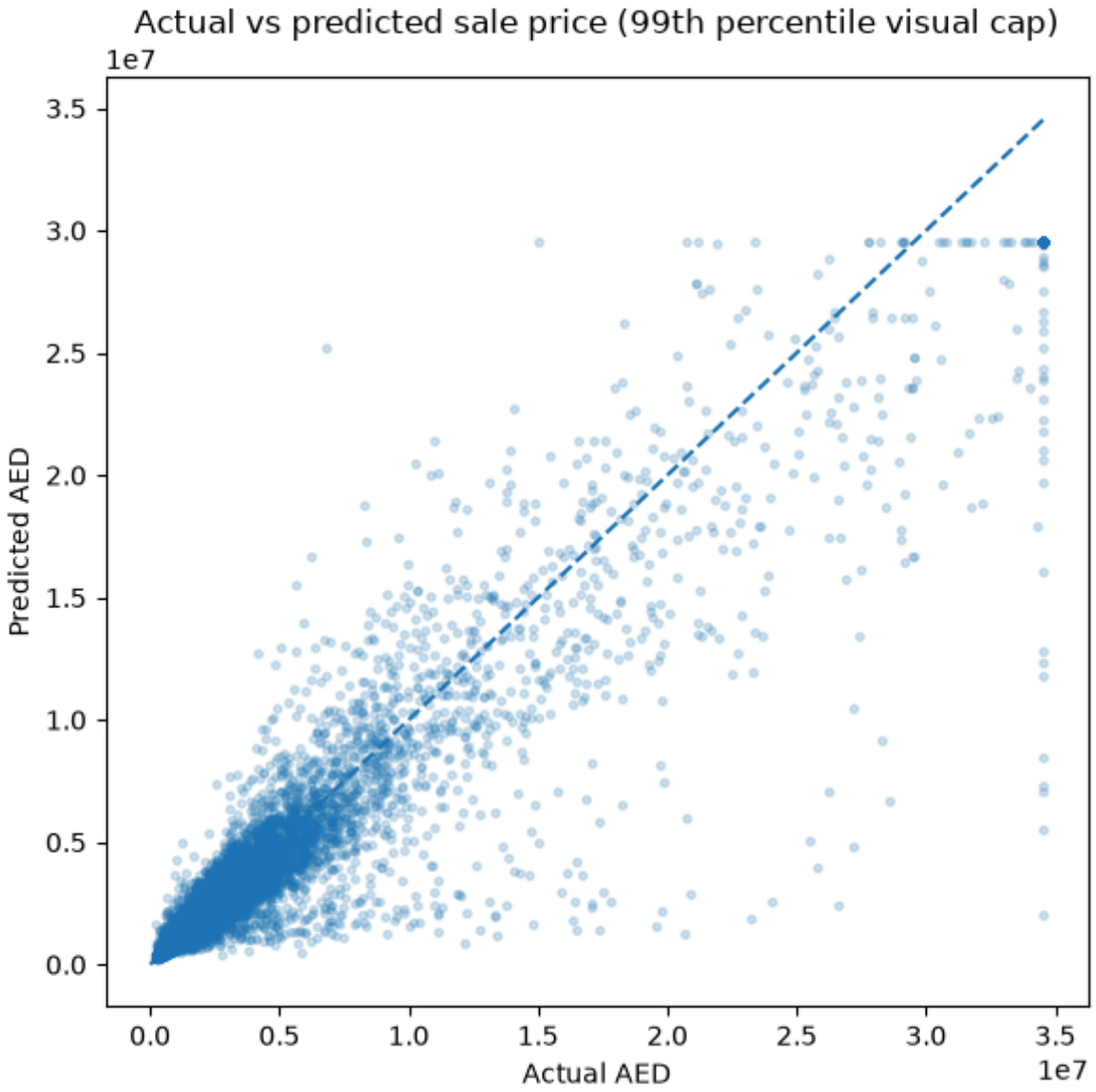
In [7]:
```
err=pd.DataFrame({'actual':y_test,'predicted':chosen_pred,'abs_error':np.abs(y_tes
err['price_band']=pd.qcut(err.actual,4,duplicates='drop')
print('Error by price band:'); display(err.groupby('price_band',observed=False).ag
print('Error by room category (top groups by sample size):'); display(err.groupby(
fig,ax=plt.subplots(figsize=(6,6)); ax.scatter(err.actual.clip(upper=err.actual.qu
```

```
Error by price band:
```

price_band	n	MAE
(176163.779, 931129.525]	4780	1.442491e+05
(931129.525, 1536792.5]	4779	2.204243e+05
(1536792.5, 2748608.22]	4779	3.664019e+05
(2748608.22, 265406177.32]	4780	2.512475e+06

Error by room category (top groups by sample size):

rooms	n	MAE
1 B/R	5034	2.213596e+05
2 B/R	4195	3.323115e+05
Studio	2781	1.626864e+05
3 B/R	2697	5.400696e+05
4 B/R	1313	8.394874e+05
5 B/R	456	1.791323e+06
PENTHOUSE	317	3.922167e+06
6 B/R	218	2.237259e+06
2 B/R+M	178	3.868775e+05
3 B/R+M	119	5.221161e+05
Single Room	80	1.111930e+06
4 B/R+M	57	7.767563e+05



7. Business recommendations and limitations

Use the model as a first-pass valuation aid, not as a replacement for a valuer. Review high value and unusual properties manually, use same community comparables, and monitor errors by community and property type. The log model can reduce the influence of the long right tail, but compare AED based test metrics before selecting it. Historical associations are not causal. The dataset lacks property condition, view, floor, precise micro location and negotiation context.

In [8]:
```
model_df.to_csv('real_estate_model_dataset.csv',index=False)
joblib.dump(chosen,'real_estate_valuation_model.joblib')
print('Saved real_estate_model_dataset.csv and real_estate_valuation_model.joblib'
```

```
Saved real_estate_model_dataset.csv and real_estate_valuation_model.joblib
```